

Memória interna

5.1 Memória principal semicondutora

- Organização
- DRAM e SRAM
- Tipos de ROM
- Lógica do chip
- Empacotamento do chip
- Organização do módulo
- Memória intercalada

5.2 Correção de erro

5.3 Organizações avançadas de DRAM

- DRAM síncrona
- DRAM RamBus
- DDR-SDRAM
- Cache DRAM

5.4 Leitura recomendada e sites Web

- Sites web recomendados

PRINCIPAIS PONTOS

- As duas formas básicas de memória de acesso aleatório semicondutora são a RAM dinâmica (DRAM) e a RAM estática (SRAM). A SRAM é mais rápida, mais cara e menos densa que a DRAM, e é usada para a memória cache. A DRAM é usada para a memória principal.
- As técnicas de correção de erro normalmente são usadas nos sistemas de memória. Estas envolvem a inclusão de bits redundantes que sejam uma função dos bits de dados, para formar um código de correção de erro. Se ocorrer um erro de bit, o código detectará e, normalmente, corrigirá o erro.
- Para compensar a velocidade relativamente baixa da DRAM, diversas organizações avançadas de DRAM foram introduzidas. As duas mais comuns são a DRAM síncrona e a DRAM RamBus. Ambas envolvem o uso do clock do sistema para proporcionar a transferência de blocos de dados.

Iniciamos este capítulo com uma análise dos subsistemas de memória principal semicondutora, incluindo memórias ROM, DRAM e SRAM. Depois, examinamos as técnicas de controle de erro usadas para melhorar a confiabilidade da memória. Em seguida, examinamos arquiteturas DRAM mais avançadas.



5.1 Memória principal semicondutora

Nos primeiros computadores, a forma mais comum de armazenamento de acesso aleatório para a memória principal do computador empregava uma matriz de loops ferromagnéticos em forma de anel, chamados de núcleos. Logo, a memória principal normalmente era chamada de núcleo (ou **core**, em inglês), um termo que persiste até hoje. Com o advento da microeletrônica, as memórias semicondutoras superaram de longe a memória de núcleo magnético. Hoje, o uso de chips semicondutores para a memória principal é quase universal. Os principais aspectos dessa tecnologia são explorados nesta seção.



Organização

O elemento básico de uma memória semicondutora é a célula de memória. Embora diversas tecnologias eletrônicas sejam utilizadas, todas as células de memória semicondutora compartilham certas propriedades:

- apresentam dois estados estáveis (ou semiestáveis), que podem ser usados para representar o binário 1 e 0;
- são capazes de ser escritas (pelo menos uma vez), para definir o estado; e
- são capazes de ser lidas, para verificar o estado.

A Figura 5.1 representa a operação de uma célula da memória. Normalmente, a célula tem três terminais funcionais, capazes de transportar um sinal elétrico. O terminal de seleção, como o nome sugere, seleciona uma célula de memória para uma operação de leitura ou escrita. O terminal de controle indica leitura ou escrita. Para a escrita, o outro terminal fornece um sinal elétrico que define o estado da célula como 1 ou 0. Para a leitura, o terminal é usado para a saída do estado da célula. Os detalhes da organização interna, funcionalidade e tempo de acesso da célula de memória dependem da tecnologia específica de circuito integrado usada e estão fora do escopo deste livro, exceto por um breve resumo. Para os nossos propósitos, vamos considerar que as células individuais podem ser selecionadas para operações de leitura e escrita.



DRAM e SRAM

Todos os tipos de memória que exploraremos neste capítulo são de acesso aleatório. Ou seja, palavras individuais da memória são acessadas diretamente por meio da lógica de endereçamento *interna*.

A Tabela 5.1 lista os principais tipos de memória de semicondutor. O mais comum é conhecido como *memória de acesso aleatório* (RAM). Este, logicamente, é um uso incorreto do termo, pois todos os tipos listados na tabela são de acesso aleatório. Uma característica distinta da RAM é a possibilidade de ler dados da memória e escrever novos dados na memória de modo fácil e rápido. Tanto a leitura quanto a escrita são realizadas por meio de sinais elétricos.

Figura 5.1 Operação da célula de memória

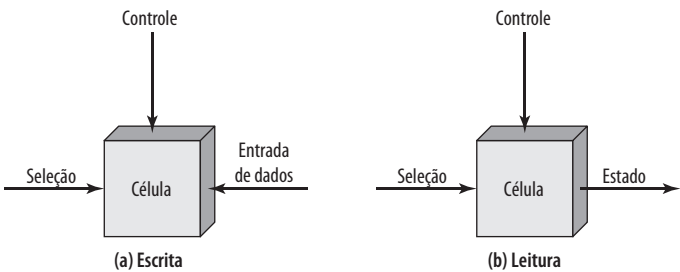


Tabela 5.1 Tipos de memória de semicondutor

Tipo de memória	Categoria	Apagamento	Mecanismo de escrita	Volatilidade
Memória de acesso aleatório (RAM)	Memória de leitura-escrita	Eletricamente, em nível de byte	Eletricamente	Volátil
Memória somente de leitura (ROM)	Memória somente de leitura	Não é possível	Máscaras	Não volátil
ROM programável (PROM, do inglês <i>programmable ROM</i>)			Eletricamente	
PROM apagável (EPROM, do inglês <i>erasable PROM</i>)	Memória principalmente de leitura	Luz UV, nível de chip		
PROM eletricamente apagável (EEPROM, do inglês <i>electrically erasable PROM</i>)		Eletricamente, nível de byte		
Memória flash		Eletricamente, nível de bloco		

Outra característica distinta da RAM é que ela é volátil. Uma RAM precisa receber uma fonte de alimentação constante. Se a energia for interrompida, os dados são perdidos. Assim, a RAM só pode ser usada como armazenamento temporário. As duas formas tradicionais de RAM usadas nos computadores são DRAM e SRAM.

RAM DINÂMICA A tecnologia da RAM é dividida em duas tecnologias: dinâmica e estática. Uma RAM dinâmica (DRAM) é feita com células que armazenam dados como carga em capacitores. A presença ou ausência de carga em um capacitor é interpretada como um binário 1 ou 0. Como os capacitores possuem uma tendência natural para descarga, as RAM dinâmicas exigem recarga periódica ("refresh" de memória) para manter o dado armazenado. O termo *dinâmica* refere-se a essa tendência de perda da carga armazenada, mesmo com energia aplicada continuamente.

A Figura 5.2a é uma estrutura de DRAM típica para uma célula individual, que armazena 1 bit. A linha de endereço é ativada quando o valor do bit dessa célula deve ser lido ou escrito. O transistor atua como uma chave que é fechada (permitindo o fluxo da corrente) se uma voltagem for aplicada à linha de endereço e é aberta (sem fluxos de corrente) se nenhuma voltagem estiver presente na linha de endereço.

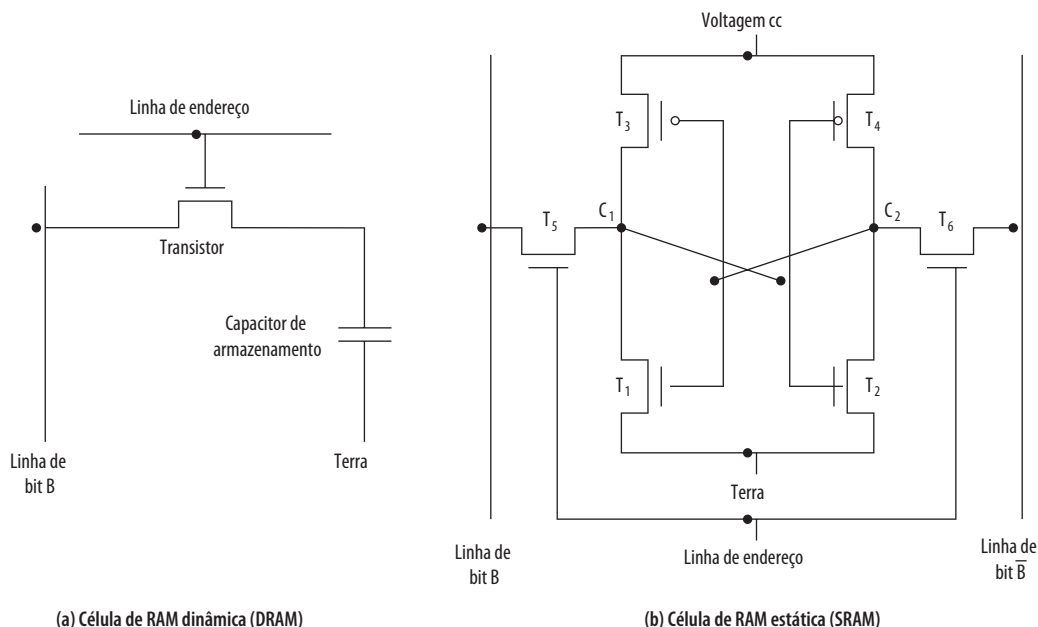
Para a operação de escrita, um sinal de tensão é aplicado à linha de bit; uma tensão alta representa 1, e uma tensão baixa representa 0. Um sinal é então aplicado à linha de endereço, permitindo que uma carga seja transferida ao capacitor.

Para a operação de leitura, quando a linha de endereço é selecionada, o transistor é ligado e a carga armazenada no capacitor é alimentada em uma linha de bit e em um amplificador. O amplificador compara a voltagem do capacitor com um valor de referência e determina se a célula contém um 1 lógico ou um 0 lógico. A leitura da célula descarrega o capacitor, que precisa ser restaurado para completar a operação.

Embora a célula da memória DRAM seja usada para armazenar um único bit (0 ou 1), é basicamente um dispositivo analógico. O capacitor pode armazenar qualquer valor de carga dentro de um intervalo; um valor de padrão determina se a carga é interpretada como 1 ou 0.

RAM ESTÁTICA Ao contrário, uma RAM estática (SRAM) é um dispositivo que usa os mesmos elementos lógicos usados no processador. Em uma SRAM, os valores binários são armazenados por meio de configurações das portas lógicas de um flip-flop tradicional (veja uma descrição dos flip-flops no Capítulo 20). Uma RAM estática manterá seus dados enquanto houver energia fornecida a ela.

Figura 5.2 Estruturas típicas de célula de memória



A Figura 5.2b é uma estrutura de SRAM típica para uma célula individual. Quatro transistores (T_1, T_2, T_3, T_4) são cruzados em um arranjo que produz um estado lógico estável. No estado lógico 1, o ponto C_1 é alto e o ponto C_2 é baixo; nesse estado, T_1 e T_4 estão desligados e T_2 e T_3 estão ligados.¹ No estado lógico 0, o ponto C_1 é baixo e o ponto C_2 é alto; nesse estado, T_1 e T_4 estão ligados e T_2 e T_3 estão desligados. Os dois estados são estáveis desde que uma voltagem de corrente contínua (cc) seja aplicada. Diferente da DRAM, nenhuma recarga é necessária para reter dados.

Assim como na DRAM, a linha de endereço da SRAM é usada para abrir ou fechar uma chave. A linha de endereço controla dois transistores (T_5 e T_6). Quando um sinal é aplicado a essa linha, os dois transistores são ligados, permitindo uma operação de leitura ou escrita. Para uma operação de leitura, o valor de bit desejado é aplicado à linha B, enquanto seu complemento é aplicado à linha $\bar{B}$. Isso força os quatro transistores (T_1, T_2, T_3, T_4) ao estado correto. Para uma operação de leitura, o valor de bit é lido da linha B.

SRAM VERSUS DRAM RAM estáticas e dinâmicas são voláteis, ou seja, a potência precisa ser fornecida continuamente à memória para preservar os valores do bit. Uma célula de memória dinâmica é mais simples e menor que uma célula de memória estática. Assim, a DRAM é mais densa (células menores = mais células por unidade de área) e mais barata que uma SRAM correspondente. Por outro lado, uma DRAM requer o suporte de um circuito de **refresh**. Para memórias maiores, o custo fixo do circuito de refresh é mais do que compensado pelo menor custo variável das células de DRAM. Assim, as DRAM tendem a ser favorecidas para requisições de grande memória. Outro ponto é que as SRAM geralmente são um pouco mais rápidas que as DRAM. Devido a essas características, a SRAM é usada para a memória cache (no chip e fora dele), e a DRAM é usada para a memória principal.



Tipos de ROM

Como o nome sugere, uma **memória somente de leitura** (ROM, do inglês *Read-Only Memory*) contém um padrão permanente de dados, que não pode ser mudado. Uma ROM é não volátil, ou seja, nenhuma fonte de energia é necessária para manter os valores dos bits na memória. Embora seja possível ler uma ROM, não é possível escrever algo novo nela. Uma aplicação importante das ROM é a microprogramação, discutida na Parte 4. Outras aplicações em potencial incluem:

- bibliotecas de funções de uso frequente;
- programas do sistema;
- tabelas de função.

Para um requisito de tamanho moderado, a vantagem da ROM é que os dados ou programa estão permanentemente na memória principal e nunca precisam ser carregados de um dispositivo de armazenamento secundário.

Uma ROM é criada como qualquer outro chip de circuito integrado, com os dados realmente gravados fisicamente no chip como parte do processo de fabricação. Isso gera dois problemas:

- A etapa de inserção de dados inclui um custo fixo relativamente grande, não importa se são fabricadas uma ou milhares de cópias de determinada ROM.
- Não há espaço para erro. Se um bit estiver errado, o lote inteiro de ROM precisa ser descartado.

Quando apenas um pequeno número de ROM com determinado conteúdo de memória é necessário, uma alternativa mais barata é a **ROM programável** (PROM). Assim como a ROM, a PROM é não volátil e pode ser escrita apenas uma vez. Para a PROM, o processo de escrita é realizado eletricamente, e pode ser realizado por um fornecedor ou cliente após a fabricação original do chip. Um equipamento especial é necessário para o processo de escrita ou “programação”. As PROM oferecem flexibilidade e conveniência. A ROM continua sendo atraente para a produção em grandes volumes.

Outra variação na memória somente de leitura é a memória principalmente de leitura, que é útil para aplicações em que operações de leitura são muito mais frequentes do que operações de escrita, mas para as quais o armazenamento não volátil é necessário. Existem três formas comuns de memória principalmente de leitura: EPROM, EEPROM e memória flash.

¹ Os círculos na ponta de T_3 e T_4 indicam negação.

A **memória somente de leitura programável e apagável** (EPROM) é lida e escrita eletricamente, assim como a PROM. Porém, antes de uma operação de escrita, todas as células de armazenamento precisam ser apagadas para retornar ao mesmo estado inicial, pela exposição do chip empacotado à radiação ultravioleta. O apagamento é feito pela exposição do chip de memória, que contém uma janela, à luz ultravioleta intensa. Esse processo de apagamento pode ser realizado repetidamente; cada apagamento pode levar até 20 minutos para ser realizado. Assim, a EPROM pode ser alterada múltiplas vezes e, como a ROM e a PROM, mantém seus dados quase indefinidamente. Para quantidades comparáveis de armazenamento, a EPROM é mais cara que a PROM, mas tem a vantagem da capacidade de múltiplas atualizações.

Uma forma mais atraente de memória principalmente de leitura é a **memória somente de leitura programável e apagável eletricamente** (EEPROM). Essa é uma memória principalmente de leitura que pode ser escrita a qualquer momento sem apagar o conteúdo anterior; somente o byte ou os bytes endereçados são atualizados. A operação de escrita leva muito mais tempo do que a operação de leitura, na ordem de muitas centenas de microssegundos por byte. A EEPROM combina a vantagem da não volatilidade com a flexibilidade de ser atualizável no local, usando as linhas comuns de controle, endereço e dados do barramento. A EEPROM é mais cara que a EPROM e também é menos densa, admitindo menos bits por chip.

Outra forma de memória de semicondutor é a **memória flash** (que tem esse nome devido à velocidade com que pode ser reprogramada). Introduzida inicialmente em meados da década de 1980, a memória flash é intermediária entre a EPROM e a EEPROM tanto no custo quanto na funcionalidade. Assim como a EEPROM, a memória flash usa uma tecnologia elétrica de apagamento. Uma memória flash inteira pode ser apagada em um ou alguns segundos, o que é muito mais rápido que a EPROM. Além disso, é possível apagar apenas blocos de memória, ao invés de um chip inteiro. A memória flash recebeu esse nome porque o microchip é organizado de modo que uma seção das células de memória é apagada em uma única ação, ou “flash”. Porém, a memória flash não oferece apagamento em nível de byte. Assim como a EPROM, a memória flash usa apenas um transistor por bit e, portanto, consegue ter a alta densidade da EPROM (em comparação com a EEPROM).



Lógica do chip

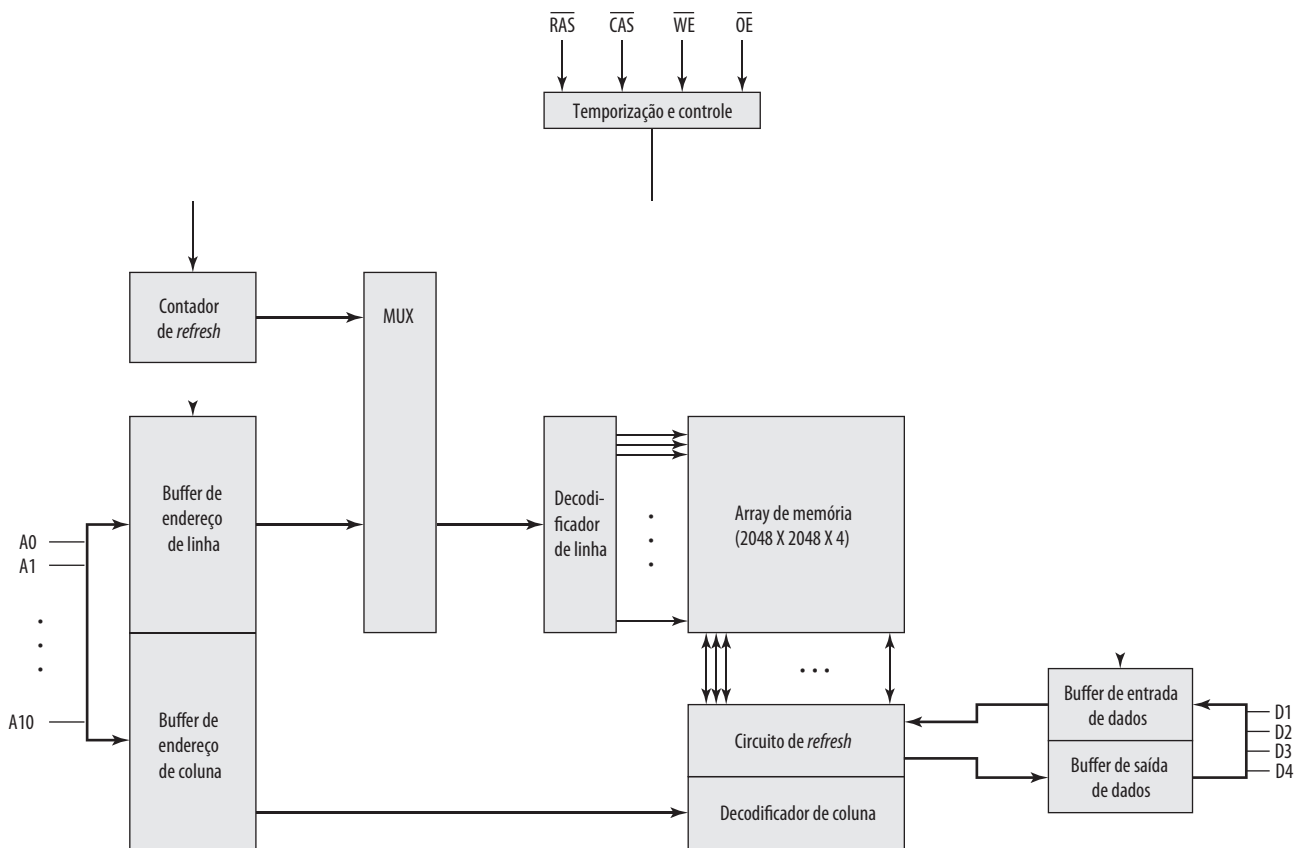
Assim como outros produtos de circuito integrado, a memória semicondutora vem em chips empacotados (Figura 2.7). Cada chip contém um array de células de memória.

Na hierarquia de memória como um todo, vimos que existem escolhas a se fazer entre velocidade, capacidade e custo. Essas escolhas também existem quando consideramos a organização das células de memória e a lógica funcional em um chip. Para memórias semicondutoras, uma das principais questões de projeto é o número de bits de dados que podem ser lidos/escritos de cada vez. Em um extremo está uma organização em que o arranjo físico das células no array é o mesmo que o arranjo lógico (percebido pelo processador) de palavras na memória. O array é organizado em W palavras de B bits cada. Por exemplo, um chip de 16 Mbits poderia ser organizado como 1 M palavras de 16 bits. No outro extremo está a chamada organização de 1 bit por chip, em que os dados são lidos/escritos 1 bit de cada vez. Vamos ilustrar a organização do chip de memória com uma DRAM; a organização da ROM é semelhante, embora mais simples.

A Figura 5.3 mostra uma organização típica de uma DRAM de 16 Mbits. Nesse caso, 4 bits são lidos ou escritos de cada vez. Logicamente, o array de memória é organizado como quatro arrays separados a cada 2048 elementos. Diversos arranjos físicos são possíveis. De qualquer forma, os elementos do array são conectados por fileiras horizontais (linha) e verticais (colunas). Cada fileira horizontal se conecta ao terminal *Select* de cada célula em sua linha; cada fileira vertical se conecta ao terminal *Data-In/Sense* de cada célula em sua coluna.

As fileiras de endereço fornecem o endereço da palavra a ser selecionada. Um total de $\log_2 W$ fileiras são necessárias. Em nosso exemplo, 11 fileiras de endereço são necessárias para selecionar uma das 2048 linhas. Essas 11 fileiras são alimentadas em um decodificador de linha, que tem 11 fileiras de entrada e 2048 fileiras de saída. A lógica do decodificador ativa uma das 2048 saídas, dependendo do padrão de bits nas 11 fileiras de entrada ($2^{11} = 2048$).

Outras 11 fileiras de endereço selecionam uma das 2048 colunas de 4 bits por coluna. Quatro linhas de dados são usadas para entrada e saída de 4 bits de e para um buffer de dados. Na entrada (escrita), o driver de bit de cada fileira é ativado para um 1 ou 0, de acordo com o valor da linha de dados correspondente. Na saída (leitura), o valor de cada linha de bit é passado por um amplificador e colocado nas 4 linhas de dados. A fileira de linha seleciona qual linha de células é usada para leitura ou escrita.

Figura 5.3 DRAM típica de 16 Megabits DRAM (4M × 4)

Como somente 4 bits são lidos/escritos nessa DRAM, é preciso haver múltiplas DRAMs conectadas ao controlador de memória para a leitura/escrita de uma palavra de dados no barramento.

Observe que existem apenas 11 linhas de endereço (A0-A10), metade do número que você esperaria para um array de 2048 × 2048. Isso é feito para economizar o número de pinos. As 22 linhas de endereço exigidas são passadas por uma lógica de seleção externa ao chip e multiplexadas nas 11 linhas de endereço. Primeiro, 11 sinais de endereço são passados ao chip para definir o endereço de linha do array, e depois os outros 11 sinais de endereço são apresentados para o endereço de coluna. Esses sinais são acompanhados por sinais de seleção de endereço de linha ($\overline{RAS}$, do inglês *Row Address Select*) e seleção de endereço de coluna ($\overline{CAS}$, do inglês *Column Address Select*) para permitir a temporização do chip.

Os pinos de habilitação de escrita ($\overline{WE}$, do inglês *Write Enable*) e habilitação de saída ($\overline{OE}$, do inglês *Output Enable*) determinam se uma operação de escrita ou leitura é realizada. Dois outros pinos, que não aparecem na Figura 5.3, são terra (V_{SS}) e uma fonte de voltagem (V_{CC}).

Como um aparte, o endereçamento multiplexado mais o uso de arrays quadrados resulta em quadruplicar o tamanho da memória com cada nova geração de chips de memória. Mais um pino dedicado ao endereçamento dobra o número de linhas e coluna e, portanto, o tamanho da memória em chip cresce por um fator de 4.

A Figura 5.3 também indica a inclusão de circuitos de *refresh*. Todas as DRAM exigem uma operação de *refresh*. Uma técnica simples de *refresh*, é desativar o chip de DRAM quando todas as células de dados são regarregadas. O contador de *refresh* percorre todos os valores de linha. Para cada linha, as linhas de saída do contador de *refresh* são fornecidas ao decodificador de linha e a linha RAS é ativada. Os dados são lidos e escritos de volta ao mesmo local. Isso faz com que cada célula na linha seja recarregada.



Empacotamento do chip

Conforme mencionamos no Capítulo 2, um circuito integrado é montado em uma cápsula que contém pinos para conexão com o mundo exterior.

A Figura 5.4a mostra um exemplo de cápsula de EPROM, que é um chip de 8 Mbits organizado como 1 M × 8. Nesse caso, a organização é tratada como uma cápsula de uma palavra por chip. A cápsula inclui 32 pinos, que é um dos tamanhos de pacote de chip padrão. Os pinos admitem as linhas de sinal apresentadas a seguir:

- O endereço da palavra sendo acessada. Para palavras de 1 M, um total de 20 ($2^{20} = 1 \text{ M}$) pinos são necessários (A0–A19).
- Os dados a serem lidos, consistindo em 8 linhas (D0–D7).
- A fonte de alimentação para o chip (V_{cc}).
- Um pino de terra (V_{ss}).
- Um pino CE. Como pode haver mais de um chip de memória, cada um conectado ao mesmo barramento de endereço, o pino CE é usado para indicar se o endereço é válido ou não para esse chip. O pino CE é ativado pela lógica conectada aos bits de mais alta ordem do barramento de endereço (ou seja, os bits de endereço acima de A19). O uso desse sinal é ilustrado atualmente.
- Uma voltagem de programa (V_{pp}) que é fornecida durante a programação (operações de escrita).

Uma configuração de pinos típica de DRAM aparece na Figura 5.4b, para um chip de 16 Mbits organizado como 4 M × 4. Existem várias diferenças quando consideramos um chip de ROM. Como uma RAM pode ser atualizada, os pinos de dados são de entrada/saída. Os pinos WE e OE indicam se essa é uma operação de escrita ou leitura. Como a DRAM é acessada por linha e coluna, e o endereço é multiplexado, somente 11 pinos de endereço são necessários para especificar as 4 M combinações de linha/coluna ($2^{11} \times 2^{11} = 2^{22} = 4 \text{ M}$). As funções dos pinos RAS e CAS já foram explicadas. Finalmente, o pino de nenhuma conexão (NC) é fornecido para que haja um número par de pinos.



Organização do módulo

Se um chip de RAM contém apenas 1 bit por palavra, então claramente precisaremos de pelo menos um número de chips igual ao número de bits por palavra. Como um exemplo, a Figura 5.5 mostra como um módulo de

Figura 5.4 Pinos e sinais do pacote de memória típico

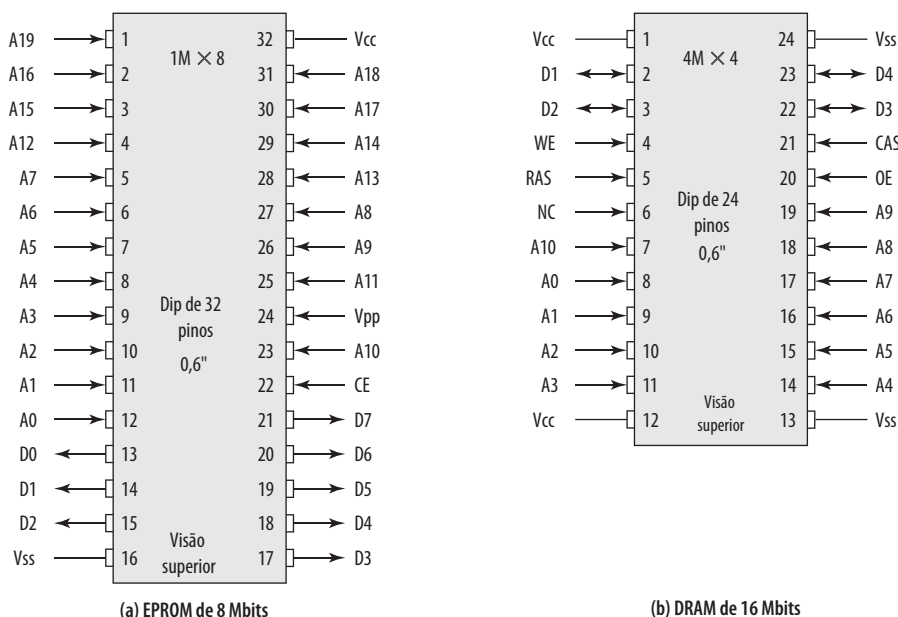
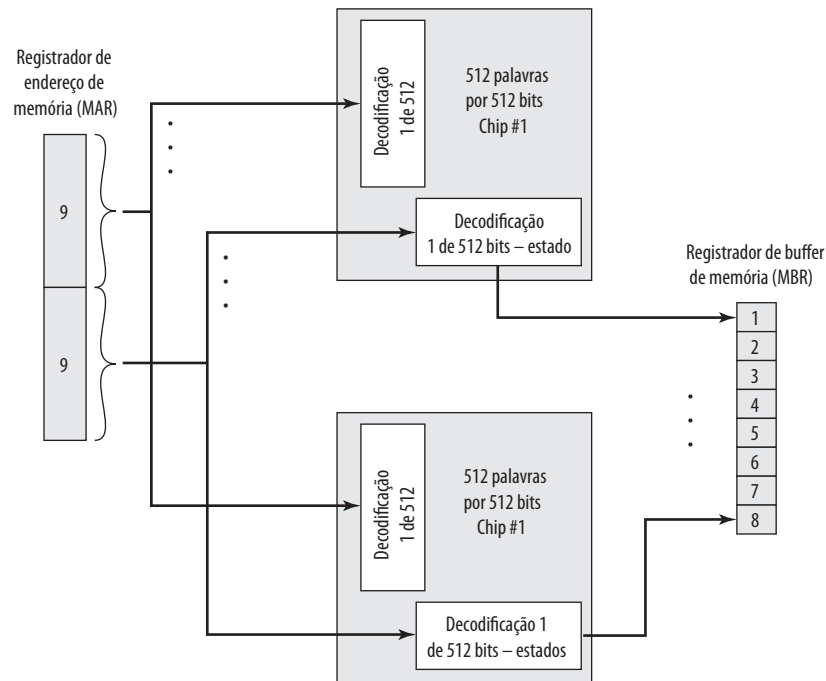


Figura 5.5 Organização de memória de 256 Kbytes

memória consistindo em 256 K palavras de 8 bits poderia ser organizado. Para palavras de 256 K, um endereço de 18 bits é necessário, sendo fornecido ao módulo a partir de alguma fonte externa (por exemplo, as linhas de endereço de um barramento ao qual o módulo está conectado). O endereço é apresentado a 8 chips de 256 K × 1 bit, cada um oferecendo a entrada/saída de 1 bit.

Essa organização funciona desde que o tamanho da memória seja igual ao número de bits por chip. No caso em que uma memória maior é necessária, um array de chips é necessário. A Figura 5.6 mostra a organização possível de uma memória consistindo em 1 M palavra por 8 bits por palavra. Nesse caso, temos quatro colunas de chips, cada coluna contendo 256 K palavras arrumadas como na Figura 5.5. Para 1 M palavra, 20 linhas de endereço são necessárias. Os 18 bits menos significativos são direcionados para todos os 32 módulos. Os 2 bits de alta ordem são entrada para um módulo lógico de seleção de grupo que envia um sinal *chip enable* a uma das quatro colunas de módulos.

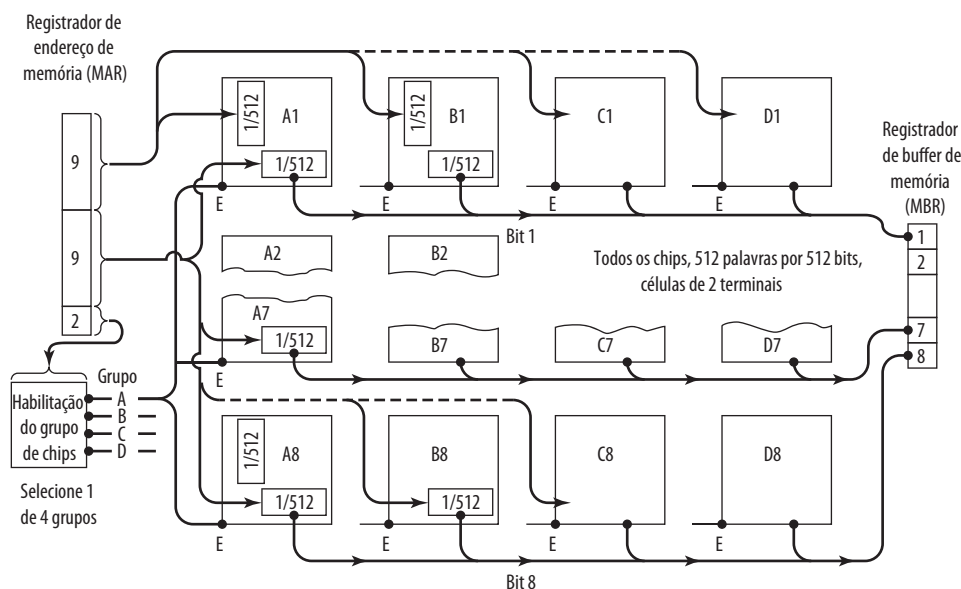


Simulador de memória intercalada



Memória intercalada

A memória principal é composta de uma coleção de chips de memória DRAM. Diversos chips podem ser agrupados para formar um **banco de memória**. É possível organizar os bancos de memória de um modo conhecido como memória intercalada (*interleaved memory*). Cada banco independentemente é capaz de atender a uma solicitação de leitura ou escrita da memória, de modo que um sistema com *K* bancos pode atender a *K* solicitações simultaneamente, aumentando as taxas de leitura ou escrita de memória por um fator de *K*. Se palavras consecutivas

Figura 5.6 Organização de memória de 1 MByte

de memória forem armazenadas em diferentes bancos, então a transferência de um bloco de memória é agilizada. O Apêndice E explora o assunto de memória intercalada.



5.2 Correção de erro

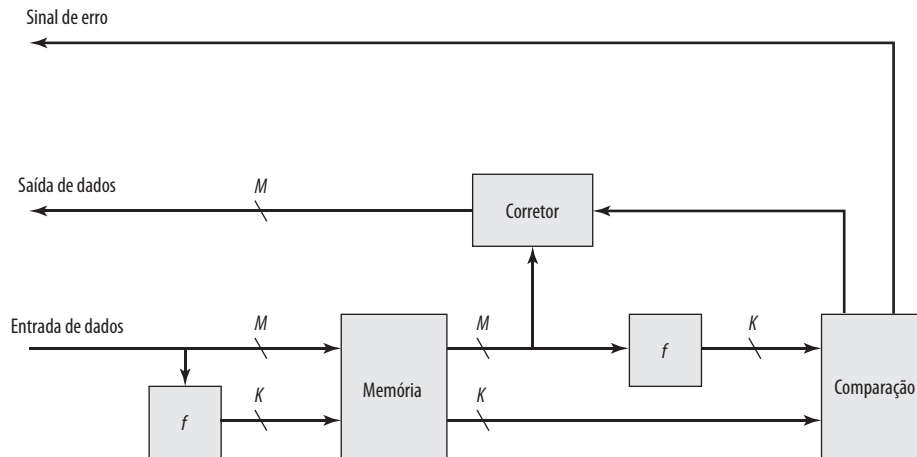
Um sistema de memória semicondutora está sujeito a erros. Estes podem ser categorizados como falhas permanentes e erros não permanentes. Uma **falha permanente** é um defeito físico permanente, de modo que a célula ou células de memória afetadas não podem armazenar dados de modo confiável, mas ficam presas em 0 ou 1, ou alternam erroneamente entre 0 e 1. Os erros permanentes podem ser causados por uso intenso em ambiente impróprio, defeitos de fabricação ou desgaste. Um **erro não permanente** é um evento aleatório, não destrutivo, que altera o conteúdo de uma ou mais células de memória sem danificar a memória. Os erros permanentes podem ser causados por problemas de fonte de alimentação ou partículas alfa. Essas partículas resultam do decaimento radioativo e são terrivelmente comuns, pois os núcleos radioativos são encontrados em pequenas quantidades em quase todos os materiais. Erros permanentes e não permanentes certamente são indesejáveis, e a maioria dos sistemas de memória modernos inclui lógica para detectar e corrigir erros.

A Figura 5.7 ilustra, em termos gerais, como o processo é executado. Quando os dados tiverem que ser lidos para a memória, um cálculo, representado como uma função f , é realizado sobre os dados para produzir um código. O código e os dados são armazenados. Assim, se uma palavra de M bits de dados tiver que ser armazenada e o código tiver K bits de tamanho, então o tamanho real da palavra armazenada é $M + K$ bits.

Quando a palavra armazenada anteriormente é lida, o código é usado para detectar e, possivelmente, corrigir erros. Um novo conjunto de K bits de código é gerado a partir dos M bits e comparado com os bits de código armazenados. A comparação gera um de três resultados:

- Nenhum erro é detectado. Os bits de dados armazenados são enviados.
- Um erro é detectado e é possível corrigi-lo. Os bits de dados mais os bits de correção de erro são alimentados em um corretor, que produz um conjunto correto de M bits a serem enviados.
- Um erro é detectado, mas não é possível corrigi-lo. Essa condição é relatada.

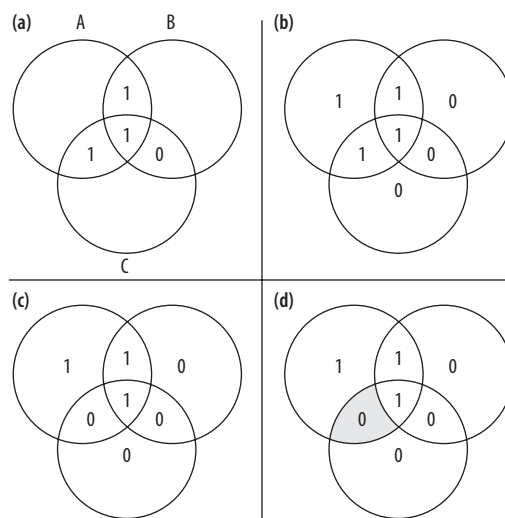
Os códigos que operam nesse padrão são conhecidos como **códigos de correção de erro**. Um código é caracterizado pelo número de erros de bit em uma palavra que ele pode corrigir e detectar.

Figura 5.7 Função do código de correção de erro

O mais simples dos códigos de correção de erro é o **código de Hamming**, idealizado por Richard Hamming, no Bell Laboratories. A Figura 5.8 usa diagramas de Venn para ilustrar o uso desse código em palavras de 4 bits ($M = 4$). Com três círculos em interseção, existem sete regiões. Atribuímos os 4 bits de dados às regiões internas (Figura 5.8a). As regiões restantes são preenchidas com o que chamamos de **bits de paridade**. Cada bit de paridade é escolhido de modo que o número total de 1s em seu círculo seja par (Figura 5.8b). Assim, como o círculo A inclui três dados 1s, o bit de paridade no círculo é definido como 1. Agora, se um erro muda um dos bits de dados (Figura 5.8c), ele é facilmente encontrado. Verificando os bits de paridade, as discrepâncias são encontradas no círculo A e no círculo C, mas não no círculo B. Somente uma das sete regiões está em A e C, mas não em B. O erro, portanto, pode ser corrigido alterando-se esse bit.

Para esclarecer os conceitos envolvidos, desenvolveremos um código que pode detectar e corrigir erros de único bit em palavras de 8 bits.

Para começar, vamos determinar o tamanho que o código deverá ter. Com referência à Figura 5.7, a lógica de comparação recebe como entrada dois valores de K bits. Uma comparação bit a bit é feita tomando-se o

Figura 5.8 Código de correção de erro de Hamming

OU-EXCLUSIVO (XOR) das duas entradas. O resultado é chamado de *palavra síndrome*. Assim, cada bit da palavra síndrome é 0 ou 1, dependendo se existe ou não uma correspondência nessa posição de bit para as duas entradas.

A palavra de síndrome, portanto, tem K bits de largura e tem um intervalo entre 0 e $2^K - 1$. O valor 0 indica que nenhum erro foi detectado, deixando $2^K - 1$ valores para indicar se houve um erro e qual bit estava com erro. Agora, como um erro poderia ocorrer em qualquer um dos M bits de dados ou K bits de verificação, precisamos ter:

$$2^K - 1 \geq M + K$$

Essa desigualdade gera o número de bits necessários para corrigir um erro de um único bit em uma palavra contendo M bits de dados. Por exemplo, para uma palavra de 8 bits de dados ($M = 8$), temos:

- $K = 3$: $2^3 - 1 < 8 + 3$;
- $K = 4$: $2^4 - 1 > 8 + 4$.

Assim, oito bits de dados exigem quatro bits de verificação. As três primeiras colunas da Tabela 5.2 listam o número de bits de verificação exigidos para diversos tamanhos de palavra de dados.

Por conveniência, gostaríamos de gerar uma palavra síndrome de 4 bits para uma palavra de dados de 8 bits com as seguintes características:

- Se a palavra síndrome contém apenas 0s, nenhum erro foi detectado.
- Se a palavra síndrome contém um e apenas um bit definido como 1, então houve um erro em um dos 4 bits de verificação. Nenhuma correção é necessária.
- Se a palavra síndrome contém mais de um bit definido como 1, então o valor numérico da palavra síndrome indica a posição do bit de dados com erro. Esse bit de dados é invertido para a correção.

Para conseguir essas características, os bits de dados e de verificação são arrumados em uma palavra de 12 bits, conforme representado na Figura 5.9. As posições de bit são numeradas de 1 a 12. Essas posições de bit cujos números de posição são potências de 2 são designadas como bits de verificação. Os bits de verificação são calculados da seguinte forma, onde o símbolo $\oplus$ designa a operação OU-EXCLUSIVO:

$$\begin{aligned} C1 &= D1 \oplus D2 \oplus D4 \oplus D5 \oplus D7 \\ C2 &= D1 \oplus D3 \oplus D4 \oplus D6 \oplus D7 \\ C4 &= D2 \oplus D3 \oplus D4 \oplus D8 \\ C8 &= D5 \oplus D6 \oplus D7 \oplus D8 \end{aligned}$$

Cada bit de verificação opera sobre todo bit de dados cujo número de posição contém um 1 na mesma posição de bit que o número de posição desse bit de verificação. Assim, todas as posições de bit de dados 3, 5, 7, 9 e 11 ($D1$, $D2$, $D4$, $D5$, $D7$) contêm um 1 no bit menos significativo do seu número de posição, assim como $C1$; as posições de bit 3, 6, 7, 10 e 11 contêm um 1 na segunda posição de bit, assim como $C2$; e assim por diante. Vendo por esse lado, a posição de bit n é verificada por aqueles bits C_i tais que $\sum i = n$. Por exemplo, a posição 7 é verificada pelos bits na posição 4, 2 e 1; e $7 = 4 + 2 + 1$.

Tabela 5.2 Aumento no tamanho da palavra com correção de erro

Bits de dados	Correção de erro único		Correção de erro único/ detecção de erro duplo	
	Bits de verificação	% de aumento	Bits de verificação	% de aumento
8	4	50	5	62,5
16	5	31,25	6	37,5
32	6	18,75	7	21,875
64	7	10,94	8	12,5
128	8	6,25	9	7,03
256	9	3,52	10	3,91

Figura 5.9 Layout de bits de dados e bits de verificação

Posição de bit	12	11	10	9	8	7	6	5	4	3	2	1
Número da posição	1100	1011	1010	1001	1000	0111	0110	0101	0100	0011	0010	0001
Bit de dados	D8	D7	D6	D5		D4	D3	D2		D1		
Bit de verificação					C8				C4		C2	C1

Vamos verificar se esse esquema funciona com um exemplo. Suponha que a palavra de entrada de 8 bits seja 00111001, com o bit de dados D1 na posição mais à direita. Os cálculos são os seguintes:

$$C1 = 1 \oplus 0 \oplus 1 \oplus 1 \oplus 0 = 1$$

$$C2 = 1 \oplus 0 \oplus 1 \oplus 1 \oplus 0 = 1$$

$$C4 = 0 \oplus 0 \oplus 1 \oplus 0 = 1$$

$$C8 = 1 \oplus 1 \oplus 0 \oplus 0 = 0$$

Suponha, agora, que o bit de dados 3 sustente um erro e seja mudado de 0 para 1. Quando os bits de verificação forem recalculados, temos:

$$C1 = 1 \oplus 0 \oplus 1 \oplus 1 \oplus 0 = 1$$

$$C2 = 1 \oplus 1 \oplus 1 \oplus 1 \oplus 0 = 0$$

$$C4 = 0 \oplus 1 \oplus 1 \oplus 0 = 0$$

$$C8 = 1 \oplus 1 \oplus 0 \oplus 0 = 0$$

Quando os novos bits de verificação forem comparados com os bits de verificação antigos, a palavra síndrome é formada:

	C8	C4	C2	C1
	0	1	1	1
$\oplus$	0	0	0	1
	0	1	1	0

O resultado é 0110, indicando que a posição de bit 6, que contém o bit de dados 3, está com erro.

A Figura 5.10 ilustra o cálculo anterior. Os bits de dados e verificação são posicionados corretamente na palavra de 12 bits. Quatro dos bits de dados têm um valor 1 (sombreado na tabela), em seus valores de posição de bit são aplicadas à função OU-EXCLUSIVO para produzir o código de Hamming 0111, que forma os quatro dígitos de verificação. O bloco inteiro que é armazenado é 001101001111. Suponha agora que o bit de dados 3, na posição de bit 6, sustente um erro e seja mudado de 0 para 1. O bloco resultante é 001101101111, com um código de

Figura 5.10 Cálculo do bit de verificação

Posição de bit	12	11	10	9	8	7	6	5	4	3	2	1
Número da posição	1100	1011	1010	1001	1000	0111	0110	0101	0100	0011	0010	0001
Bit de dados	D8	D7	D6	D5		D4	D3	D2		D1		
Bit de verificação					C8				C4		C2	C1
Palavra armazenada como	0	0	1	1	0	1	0	0	1	1	1	1
Palavra buscada como	0	0	1	1	0	1	1	0	1	1	1	1
Número da posição	1100	1011	1010	1001	1000	0111	0110	0101	0100	0011	0010	0001
Bit de verificação					0				0		0	1

Hamming de 0111. Um XOR do código de Hamming e todos os valores de posição de bit para bits de dados não zero resulta em 0110. O resultado diferente de zero detecta um erro e indica que o erro está na posição de bit 6.

O código que descrevemos é conhecido como um código de **correção de único erro** (SEC, do inglês *single-error-correcting*). Normalmente, a memória semicondutora é equipada com um código de correção de único erro, detecção de duplo erro (SEC-DED, do inglês *double-error-detecting*). Conforme mostra a Tabela 5.2, esses códigos exigem um bit adicional em comparação com os códigos SEC.

A Figura 5.11 ilustra como esse código funciona, novamente com uma palavra de dados de 4 bits. A sequência mostra que, se houver dois erros (Figura 5.11c), o procedimento de verificação se perde (d) e piora o problema, criando um terceiro erro (e). Para contornar o problema, um oitavo bit é acrescentado, definido de modo que o número total de 1s no diagrama seja par. O bit de paridade extra detecta o erro (f).

Um código de correção de erro melhora a confiabilidade da memória ao custo de maior complexidade. Com uma organização de 1 bit por chip, um código SEC-DED geralmente é considerado adequado. Por exemplo, as implementações IBM 30xx usavam um código SEC-DED de 8 bits para cada 64 bits de dados na memória principal. Assim, o tamanho da memória principal é, na realidade, cerca de 12% maior do que fica disponível ao usuário. Os computadores VAX usavam um SEC-DED de 7 bits para cada 32 bits de memória, para um *overhead* de 22%. Uma série de DRAM modernas utilizam 9 bits de verificação para cada 128 bits de dados, com um *overhead* de 7% (Sharma, 1997^o).



5.3 Organizações avançadas de DRAM

Conforme discutimos no Capítulo 2, um dos gargalos mais críticos do sistema quando se usam processadores de alto desempenho é a interface com a memória interna principal. Essa interface é o caminho mais importante no sistema de computação inteiro. O bloco de montagem básico da memória principal continua sendo o chip de DRAM, como tem sido há décadas; até pouco tempo, não houve mudanças significativas na arquitetura da DRAM desde o início da década de 1970. O chip de DRAM tradicional é restrito tanto por sua arquitetura interna quanto por sua interface com o barramento de memória do processador.

Vimos que uma abordagem para melhorar o problema do desempenho da memória principal DRAM tem sido inserir um ou mais níveis de cache SRAM de alta velocidade entre a memória principal DRAM e o processador. Mas a SRAM é muito mais cara que a DRAM, e a expansão do tamanho da cache além de um certo ponto gera retornos decrescentes.

Nos anos recentes, diversas melhorias na arquitetura básica da DRAM foram exploradas, e algumas destas agora estão no mercado. Os esquemas que atualmente dominam o mercado são SDRAM, DDR-DRAM e RDRAM.

Figura 5.11 Código SEC-DEC de Hamming

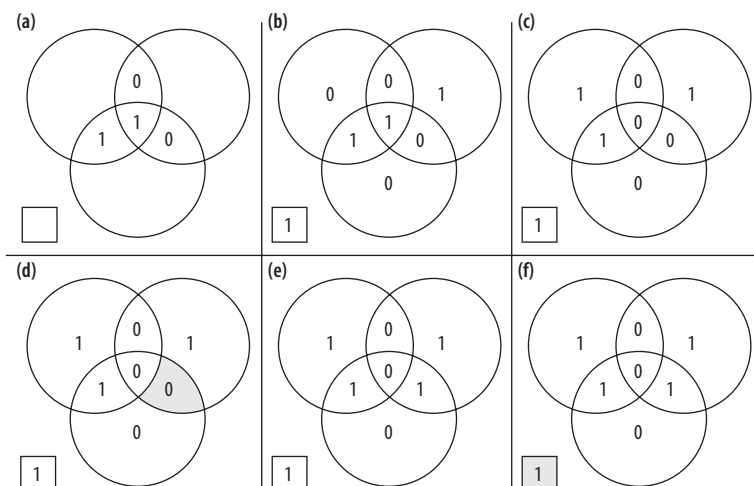


Tabela 5.3 Comparação do desempenho de algumas alternativas à DRAM

	Frequência de clock (MHz)	Taxa de transferência (GB/s)	Tempo de acesso (ns)	Contagem de pinos
SDRAM	166	1,3	18	168
DDR	200	3,2	12,5	184
RDRAM	600	4,8	12	162

A Tabela 5.3 oferece uma comparação de desempenho. CDRAM também recebeu alguma atenção. Examinaremos cada uma dessas técnicas nesta seção.



DRAM síncrona

Uma das formas mais utilizadas de DRAM é a DRAM síncrona (SDRAM, do inglês *synchronous DRAM*) (Vogley, 1994^b). Diferente da DRAM tradicional, que é assíncrona, a SDRAM troca dados com o processador sincronizado com um sinal de clock externo e executando na velocidade plena do barramento do processador/memória, sem imposição de estados de espera.

Em uma DRAM típica, o processador apresenta endereços e níveis de controle à memória, indicando que um conjunto de dados em determinado local na memória deve ser lido da DRAM ou escrito nela. Após uma espera, isto é, o tempo de acesso, a DRAM escreve ou lê os dados. Durante o atraso do tempo de acesso, a DRAM realiza diversas funções internas, como ativar a alta capacitância das fileiras de linha e coluna, verificar os dados e roteá-los através dos buffers de saída. O processador deve simplesmente esperar por esse atraso, diminuindo o desempenho do sistema.

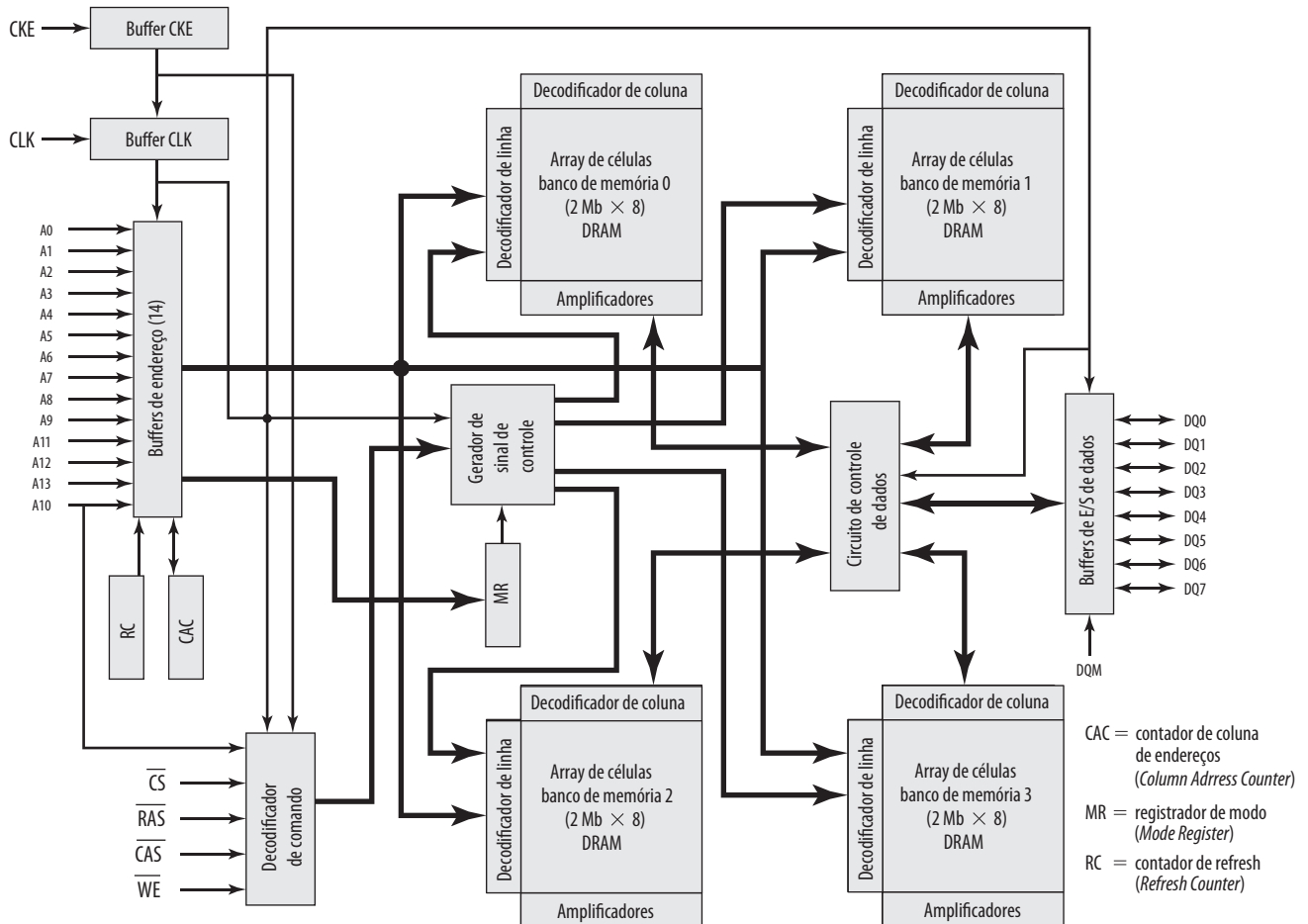
Com o acesso síncrono, a DRAM move dados para dentro e para fora sob o controle do clock do sistema. O processador ou outro mestre emite a instrução e informação de endereço, que é travada pela DRAM. A DRAM, então, responde após determinado número de ciclos de clock. Nesse meio-tempo, o mestre pode seguramente realizar outras tarefas enquanto a SDRAM está processando a solicitação.

A Figura 5.12 mostra a lógica interna da SDRAM de 64 Mb da IBM (IBM, 2001^c), que é típico da organização da SDRAM, e a Tabela 5.4 define as diversas atribuições de pino.

A SDRAM emprega um modo de rajada (*burst*) para eliminar o tempo de configuração de endereço e tempo de pré-carga de fileira de linha e coluna após o primeiro acesso. No modo burst, uma série de bits de dados pode ser enviada rapidamente após o primeiro bit ter sido acessado. Esse modo é útil quando todos os bits a serem acessados estiverem na sequência e na mesma linha do array que o acesso inicial. Além disso, a SDRAM tem uma arquitetura interna de banco múltiplo, que melhora oportunidades para paralelismo no chip.

Tabela 5.4 Atribuições de pino da SDRAM

A0 a A13	Entradas de endereço
CLK	Entrada de clock
CKE	Habilitação de clock
$\overline{CS}$	Seleção de chip
$\overline{RAS}$	Strobe de endereço de linha
$\overline{CAS}$	Strobe de endereço de coluna
$\overline{WE}$	Habilitação de escrita
DQ0 a DQ7	Entrada/saída de dados
DQM	Máscara de dados

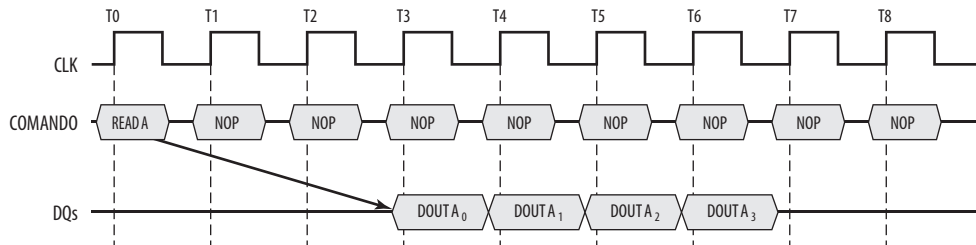
Figura 5.12 DRAM síncrona (SDRAM)

O registrador de modo (MR, do inglês *mode register*) e a lógica de controle associada é outro recurso fundamental que diferencia as SDRAM das DRAM convencionais. Ele oferece um mecanismo para personalizar a SDRAM para se ajustar às necessidades específicas do sistema. O registrador de modo especifica o tamanho da rajada, que é o número de unidades de dados separadas alimentadas sincronamente no barramento. O registrador também permite que o programador ajuste a latência entre o recebimento de uma solicitação de leitura e o início da transferência de dados.

A SDRAM funciona melhor quando está transferindo grandes blocos de dados de forma serial, como para aplicações como processamento de textos, planilhas e multimídia.

A Figura 5.13 mostra um exemplo de operação da SDRAM. Nesse caso, o tamanho de rajada é 4 e a latência é 2. O comando de leitura de rajada é iniciado com $\overline{\text{CS}}$ e $\overline{\text{CAS}}$ baixos, enquanto se mantém $\overline{\text{RAS}}$ e $\overline{\text{WE}}$ altos na transição de subida do clock. As entradas de endereço determinam o endereço de coluna inicial para a rajada, e o registrador de modo define o tipo da rajada (sequencial ou intercalada) e o tamanho da rajada (1, 2, 4, 8, página completa). O atraso a partir do início do comando até quando os dados da primeira célula aparecem nas saídas é igual ao valor da latência de $\overline{\text{CAS}}$ que é definida no registrador de modo.

Há agora uma versão avançada da SDRAM, conhecida como *Double Data Rate SDRAM* (DDR-SDRAM), que resolve a limitação de uma vez por ciclo. A DDR-SDRAM pode enviar dados ao processador duas vezes por ciclo de clock.

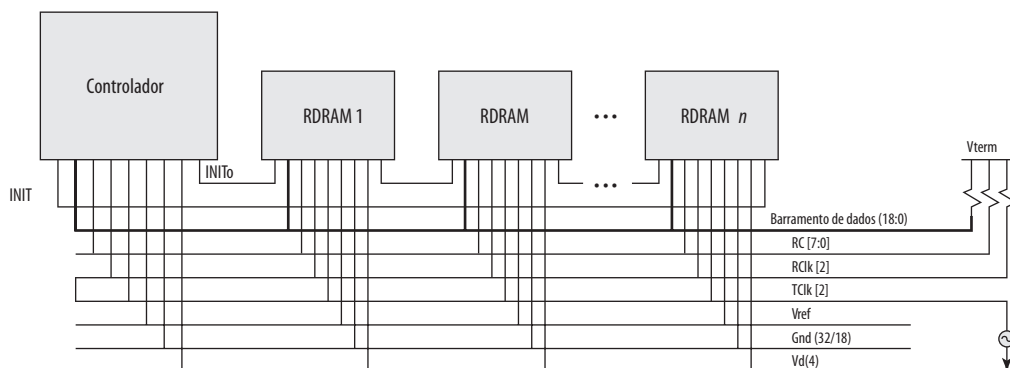
Figura 5.13 Temporização de leitura da SDRAM (tamanho de rajada = 4, latência de $\overline{\text{CAS}} = 2$)

DRAM Rambus

A RDRAM, desenvolvida pela Rambus (Farmwald, 1999;^dCrisp. 1997^e), tem sido adotada pela Intel para os seus processadores Pentium e Itanium. Ela se tornou o concorrente principal da SDRAM. Chips RDRAM são encapsuladas verticalmente, com todos os pinos em um lado. O chip troca dados com o processador por 28 fios com não mais do que 12 centímetros de extensão. O barramento pode endereçar até 320 chips de RDRAM a uma taxa de 1,6 GBps.

O barramento RDRAM especial oferece informações de endereço e controle usando um protocolo assíncrono, orientado a bloco. Após um tempo de acesso inicial de 480 ns, isso produz a taxa de dados de 1,6 GBps. O que torna essa velocidade possível é o próprio barramento, que define impedâncias, clocking e sinais com muita precisão. Em vez de ser controlada por sinais RAS, CAS, R/W e CE usados nas DRAM convencionais, uma RDRAM recebe uma solicitação de memória pelo barramento de alta velocidade. Essa solicitação contém o endereço desejado, o tipo de operação e o número de bytes na operação.

A Figura 5.14 ilustra o layout da RDRAM. A configuração consiste em um controlador e uma série de módulos de RDRAM conectados por um barramento comum. O controlador está em uma extremidade da configuração e o canto oposto do barramento é o fim das linhas de barramento. O barramento inclui 18 linhas de dados (16 dados reais, dois de paridade) pulsando no dobro da velocidade do clock; ou seja, 1 bit é enviado em cada transição do sinal de clock. Isso resulta em uma taxa de sinal em cada linha de dados de 800 Mbps. Existe um conjunto separado de 8 linhas (RC) usadas para sinais de endereço e controle. Há também um sinal de clock que começa na extremidade oposta do controlador, propaga-se até o extremo do controlador e depois retorna. Um módulo de RDRAM envia dados ao controlador de forma síncrona ao clock para o mestre, e o controlador envia dados a uma RDRAM de forma síncrona com o sinal de clock na direção oposta. As linhas de barramento restantes incluem uma tensão de referência, terra e fonte de alimentação.

Figura 5.14 Estrutura da RDRAM



DDR-SDRAM

A SDRAM é limitada pelo fato de que só pode enviar dados ao processador uma vez por ciclo de clock do barramento. Uma nova versão da SDRAM, conhecida como *Double Data Rate* SDRAM (DDR-SDRAM), pode enviar dados duas vezes por ciclo de clock, uma vez na transição de subida do pulso de clock e uma vez na transição de descida.

A DDR-SDRAM foi desenvolvida pela JEDEC *Solid State Technology Association*, a agência de padronização da engenharia de semicondutor da *Electronic Industries Alliance*. Diversas empresas fabricam chips DDR, que são bastante usados nos computadores desktop e servidores.

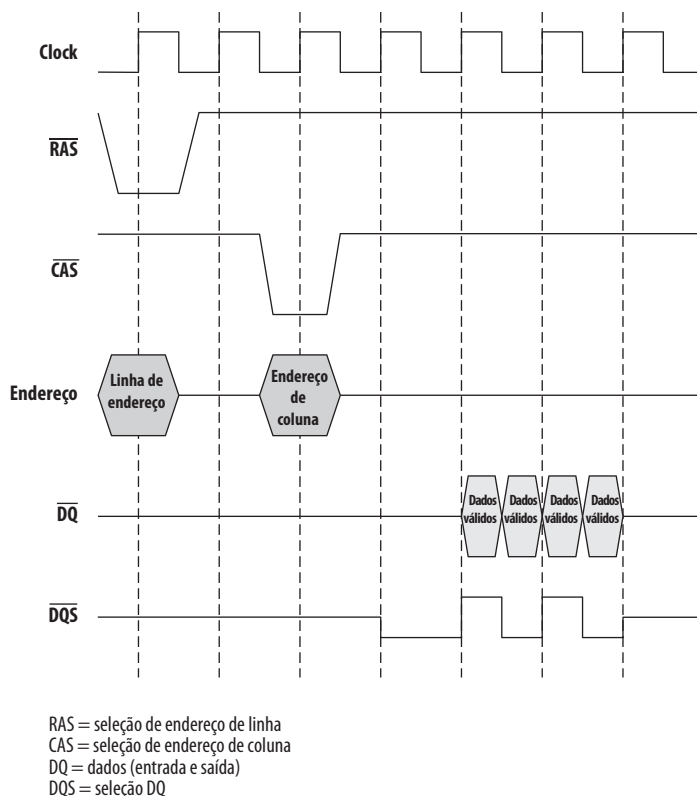
A Figura 5.15 mostra a temporização básica para uma leitura DDR. A transferência de dados é sincronizada com a transição de subida e descida do clock. Ela também é sincronizada com um sinal de strobe de dados bi-direcional (DQS) que é fornecido pelo controlador de memória durante uma leitura e pela DRAM durante uma escrita. Nas implementações típicas, o DQS é ignorado durante a leitura. Uma explicação do uso do DQS nas escritas está fora do nosso escopo; veja mais detalhes em Jacob, Ng e Wang (2008^f).

Existem duas gerações de melhoria na tecnologia DDR. DDR2 aumenta a taxa de transferência de dados aumentando a frequência operacional do chip de RAM e aumentando o buffer de pré-busca de 2 bits para 4 bits por chip. O buffer de pré-busca é uma cache de memória localizada no chip de RAM. O buffer permite que o chip de RAM pré-posicione os bits a serem colocados na base de dados o mais rapidamente possível. A DDR3, introduzida em 2007, aumenta o tamanho do buffer de pré-busca para 8 bits.

Teoricamente, um módulo DDR pode transferir dados a uma taxa de clock na faixa de 200 a 600 MHz; um módulo DDR2 transfere a uma taxa de clock de 400 a 1066 MHz; e um módulo DDR3 transfere a uma taxa de clock de 800 a 1600 MHz. Na prática, taxas um pouco menores são alcançadas.

O Apêndice K oferece mais detalhes sobre a tecnologia DDR.

Figura 5.15 Temporização de leitura da SDRAM DDR





Cache DRAM

A cache DRAM (CDRAM, do inglês *cache DRAM*), desenvolvida pela Mitsubishi (Hidaka et al., 1990^a; Zhang, Zhu e Zhang, 2001^b), integra uma cache SRAM pequena (16 Kb) a um chip de DRAM genérico.

A SRAM na CDRAM pode ser vista de duas maneiras. Primeiro, ela pode ser usada como uma cache verdadeira, consistindo em uma série de linhas de 64 bits. O modo de cache da CDRAM é eficaz para o acesso aleatório normal à memória.

A SRAM na CDRAM também pode ser usada como um buffer para dar suporte ao acesso serial de um bloco de dados. Por exemplo, para renovar uma tela de mapa de bits, a CDRAM pode previamente buscar os dados da DRAM no buffer de SRAM. Os acessos subsequentes ao chip resultam em acesso unicamente à SRAM.



5.4 Leitura recomendada e sites Web

Prince (1997ⁱ) oferece um tratamento abrangente das tecnologias de memória semicondutoras, incluindo SRAM, DRAM e memórias flash. Sharma (1997^a) cobre o mesmo material, com mais ênfase nas questões de teste e confiabilidade. Sharma (2003^j) e Prince (2002^k) mostram as arquiteturas de DRAM e SRAM avançadas. Para ter uma visão profunda sobre DRAM, consulte Jacob, Ng e Wang (2008^l) e Keeth e Baker (2001^l). Cuppu et al. (2001^m) oferece uma comparação de desempenho interessante dos diversos esquemas de DRAM. Bez et al. (2003ⁿ) é uma introdução abrangente da tecnologia de memória flash.

Uma boa explicação dos códigos de correção de erro está contida em McEliece (1985^o). Para obter um estudo mais profundo, algumas abordagens em livro, que valem a pena ser lidas, são Adamek (1991^p) e Blahut (1983^q). Um tratamento teórico e matemático legível sobre códigos de correção de erro é Ash (1990^r). Sharma (1997^a) contém um bom estudo sobre códigos usados nas memórias principais contemporâneas.



Sites Web recomendados

The RAM Guide: boa introdução à tecnologia de RAM mais uma série de links úteis.

RDRAM: outro site útil para informações sobre RDRAM.

Principais termos, perguntas de revisão e problemas

Principais termos

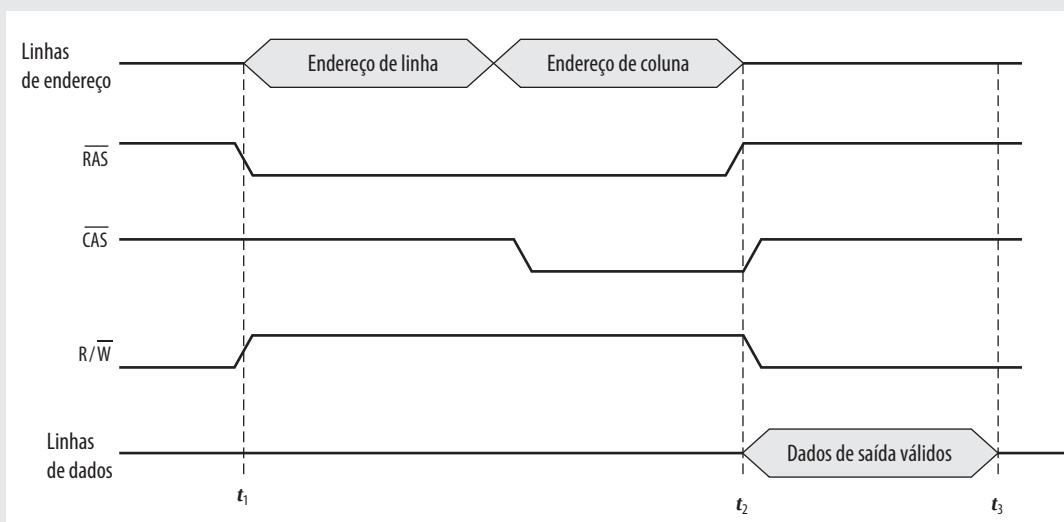
Cache DRAM (CDRAM)	Falha permanente	Código de correção de único erro, detecção de duplo erro (SEC-DED)
RAM dinâmica (DRAM)	Memória não volátil	Erro não permanente
ROM programável apagável eletricamente (EEPROM)	ROM programável (PROM)	RAM estática (SRAM)
ROM programável apagável (EPROM)	DRAM RamBus (RDRAM)	DRAM síncrona (SDRAM)
Código de correção de erro (ECC)	Memória principalmente de leitura	Palavra síndrome
Correção de erro	Memória semicondutora	Memória volátil
Memória flash	Código de correção de único erro (SEC)	
Código de Hamming		

Perguntas de revisão

- 5.1 Quais são as principais propriedades da memória semicondutora?
- 5.2 Quais são os dois sentidos em que o termo *memória de acesso aleatório* é usado?
- 5.3 Qual é a diferença entre DRAM e SRAM em termos de aplicação?
- 5.4 Qual é a diferença entre DRAM e SRAM em termos das características como velocidade, tamanho e custo?
- 5.5 Explique por que um tipo de RAM é considerado como analógico e o outro digital.
- 5.6 Quais são algumas aplicações para a ROM?
- 5.7 Quais são as diferenças entre EPROM, EEPROM e memória flash?
- 5.8 Explique a função de cada pino na Figura 5.4b.
- 5.9 O que é bit de paridade?
- 5.10 Como é interpretada a palavra síndrome para o código de Hamming?
- 5.11 Como a SDRAM difere de uma DRAM comum?

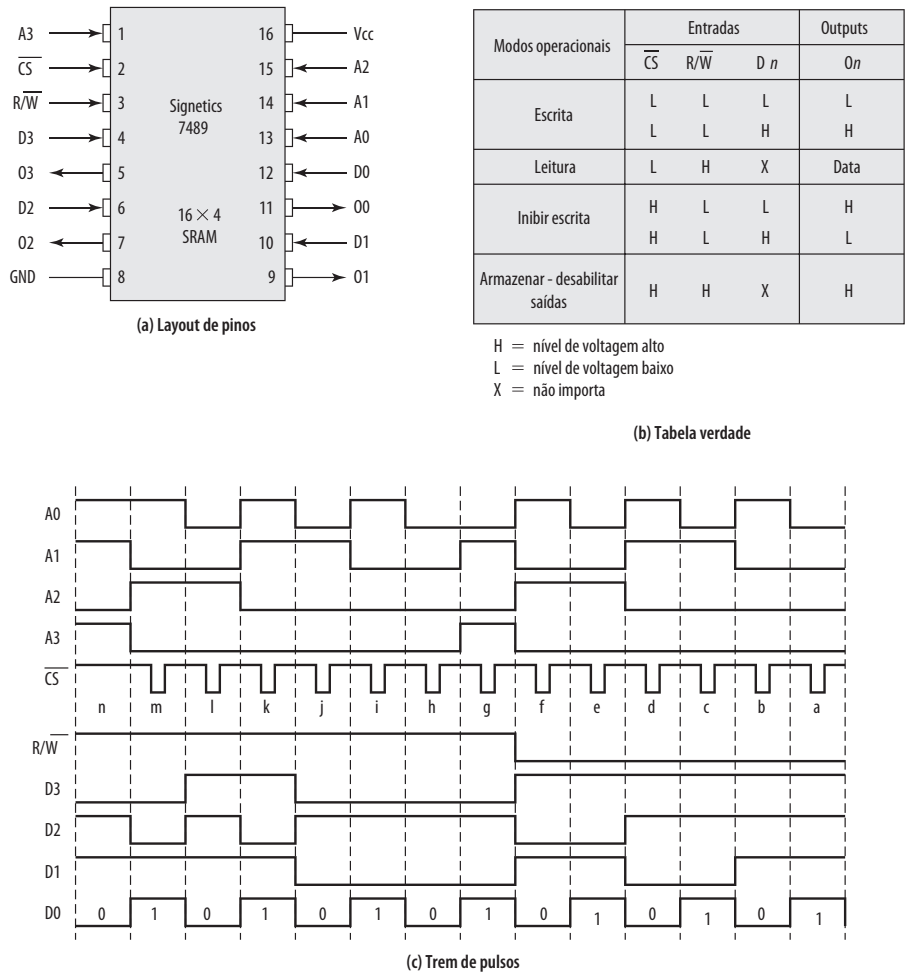
Problemas

- 5.1 Sugira motivos pelos quais as RAM têm sido tradicionalmente organizadas como 1 bit por chip, enquanto as ROM normalmente são organizadas com múltiplos bits por chip.
- 5.2 Considere uma RAM dinâmica que precisa ter um ciclo de refresh de 64 vezes por ns. Cada operação de refresh exige 150 ns; um ciclo de memória exige 250 ns. Que percentagem do tempo de operação total da memória precisa ser dado aos circuitos de refresh?
- 5.3 A Figura 5.16 mostra um diagrama de temporização simplificado para uma operação de leitura de DRAM por um barramento. O tempo de acesso é considerado de t_1 a t_2 . Então, existe um tempo de recarga, durando de t_2 a t_3 , durante o qual os chips de DRAM terão que ser recarregados antes que o processador possa acessá-los novamente.
 - a. Assuma que o tempo de acesso é de 60 ns e o tempo de recarga é 40 ns. Qual é o tempo de ciclo da memória? Qual é o valor máximo de dados que essa DRAM pode sustentar, assumindo que temos 1 bit de saída?
 - b. Construindo um sistema com 32 bits de memória usando esses chips, qual será o valor de transferência de dados?
- 5.4 A Figura 5.6 indica como construir um módulo de chips que pode armazenar 1 MByte com base em um grupo de quatro chips de 256 Kbytes. Digamos que esse módulo de chips seja encapsulado como um único chip de 1 MByte, onde o tamanho da palavra é de 1 byte. Dê um diagrama de chip de alto nível de como construir uma memória de computador de 8 MBytes usando oito chips de 1 MByte. Não se esqueça de mostrar as linhas de endereços no seu diagrama e mostrar para que são usadas as linhas de endereço.

Figura 5.16 Temporização de leitura de DRAM simplificada

- 5.5 Em um sistema típico baseado no Intel 8086, conectado via barramento do sistema à memória DRAM, para uma operação de leitura, $\overline{RAS}$ é ativado pela transição final do sinal Address Enable (Figura 3.19). Porém, devido à propagação e outros atrasos, $\overline{RAS}$ não é ativo até 50 ns após Address Enable retornar para o estado baixo. Suponha que esse último ocorra no meio da segunda metade do estado T_1 (um pouco antes do que na Figura 3.19). Os dados são lidos pelo processador ao final de T_3 . Contudo, para que o processador possa receber os dados corretamente, esses dados devem ser fornecidos 60 ns antes pela memória. Esse intervalo leva em conta os atrasos de propagação ao longo dos caminhos de dados (da memória ao processador) e os requisitos de hold time dos dados para o processador. Considere uma frequência de clock de 10 MHz.
- a. Que velocidade (tempo de acesso) as DRAM devem ter se nenhum estado de espera tiver que ser inserido?
 - b. Quantos estados de espera temos que inserir por operação de leitura da memória se o tempo de acesso das DRAM for 150 ns?
- 5.6 A memória de um microcomputador em particular é montada a partir de DRAM 64 K $\times$ 1. De acordo com o manual da memória, o array de células da DRAM é organizado em 256 linhas. Cada linha precisa ter o refresh pelo menos uma vez a cada 4 ms. Suponha que se faça refresh a memória em uma base estritamente periódica.
- a. Qual é o período entre as solicitações de refresh sucessivas?
 - b. Por quanto tempo precisamos de um contador de endereço de refresh?
- 5.7 A Figura 5.17 mostra uma das primeiras SRAM, o chip Signetics 7489 de 16 $\times$ 4, que armazena 16 palavras de 4 bits.
- a. Liste o modo de operação do chip para cada pulso de entrada $\overline{CS}$ mostrado na Figura 5.17c.
 - b. Liste o conteúdo de memória dos locais de palavra de 0 a 6 após o pulso n.
 - c. Qual é o estado dos terminais de dados de saída para os pulsos de entrada de h até m?

Figura 5.17 A SRAM Signetics 7489



- 5.8** Projete uma memória de 16 bits com capacidade total de 8 192 bits usando chips de SRAM de tamanho 64×1 bit. Dê a configuração de array dos chips na placa de memória mostrando todos os sinais de entrada e saída exigidos para atribuir essa memória ao espaço de endereço mais baixo. O projeto deve permitir acessos de byte e palavra de 16 bits.
- 5.9** Uma unidade de medida comum para taxas de falha de componentes eletrônicos é a **unidade de falha** (FIT, do inglês *Failure unit*), expressa como a taxa de falhas por bilhão de horas do dispositivo. Outra medida bem conhecida, porém pouco usada, é o **tempo médio entre falhas** (MTBF, do inglês *Mean Time Between Failures*), que é o tempo médio de operação de determinado componente até que ele falhe. Considere uma memória de 1 MB de um microprocessador de 16 bits com $256 \text{ K} \times 1$ DRAM. Calcule seu MTBF supondo que 2 000 FIT para cada DRAM.
- 5.10** Para o código de Hamming mostrado na Figura 5.10, mostre o que acontece quando um bit de verificação, ao invés de um bit de dados, tem um erro.
- 5.11** Suponha que uma palavra de dados de 8 bits armazenada na memória seja 11000010. Usando o algoritmo de Hamming, determine quais bits de verificação seriam armazenados na memória com a palavra de dados. Mostre como você chegou a sua resposta.
- 5.12** Para uma palavra de 8 bits 00111001, os bits de verificação armazenados com ela seriam 0111. Suponha, quando a palavra for lida da memória, que os bits de verificação são calculados como 1101. Qual palavra de dados foi lida da memória?
- 5.13** Quantos bits de verificação são necessários se o código de correção de erro de Hamming for usado para detectar erros de único bit em uma palavra de dados de 1024 bits?
- 5.14** Desenvolva um código SEC para uma palavra de dados de 16 bits. Gere o código para a palavra de dados 010100000111001. Mostre que o código identificará corretamente um erro no bit de dados 5.

Referências

- a SHARMA, A. *Semiconductor memories: technology, testing, and reliability*. Nova York: IEEE Press, 1997.
- b VOGLEY, B. "800 megabyte per second systems via use of synchronous DRAM". *Proceedings, COMPCON '94*, mar. 1994.
- c International Business Machines, Inc. *64 Mb Synchronous DRAM*. IBM Data Sheet 364164, jan. 2001.
- d FARMWALD, M. e MOORING, D. "A fast path to one memory". *IEEE Spectrum*, out. 1992.
- e CRISP, R. "Direct RAMBUS technology: the new main memory standard". *IEEE Micro*, nov./dez. 1997.
- f JACOB, B.; Ng, S.; e Wang, D. *Memory systems: cache, DRAM, disk*. Boston: Morgan Kaufmann, 2008.
- g HIDAKA, H.; MATSUDA, Y.; ASAKURA, M. e KAZUYASU, F. "The cache DRAM architecture: A DRAM with an on-chip cache memory". *IEEE Micro*, abr. 1990.
- h ZHANG, Z.; ZHU, Z. e ZHANG, X. "Cached DRAM for ILP processor memory access latency reduction". *IEEE Micro*, jul./ago. 2001.
- i PRINCE, B. *Semiconductor memories*. Nova York: Wiley, 1997.
- j SHARMA, A. *Advanced semiconductor memories: architectures, designs, and applications*. New York: IEEE Press, 2003.
- k PRINCE, B. *Emerging memories: technologies and trends*. Norwell, MA: Kluwer, 2002.
- l KEETH, B. e BAKER, R. *DRAM circuit design: a tutorial*. Piscataway, NJ: IEEE Press, 2001.
- m CUPPU, V, et al. "High performance DRAMS in workstation environments". *IEEE Transactions on Computers*, nov. 2001.
- n BEZ, R.; et al. "Introduction to flash memory". *Proceedings of the IEEE*, abr. 2003.
- o MCELIECE, R. "The reliability of computer memories". *Scientific American*, jan. 1985.
- p ADAMEK, J. *Foundations of coding*. Nova York: Wiley, 1991.
- q BLAHUT, R. *Theory and practice of error control codes*. Reading, MA: Addison-Wesley, 1983.
- r ASH, R. *Information theory*. Nova York: Dover, 1990.